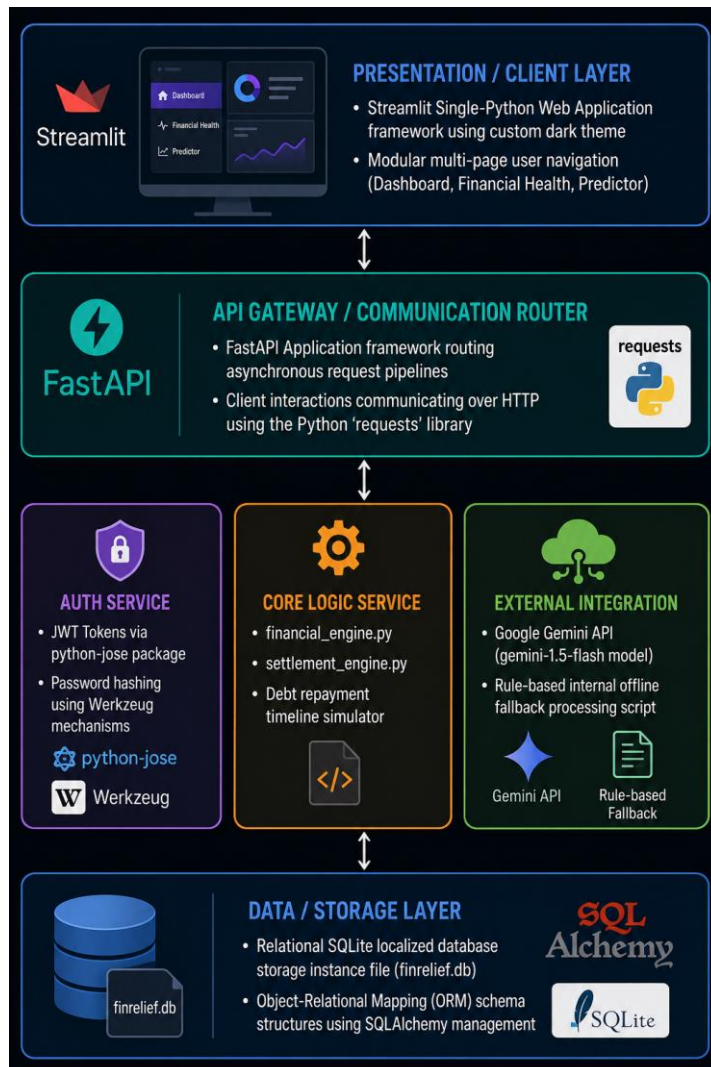


Solution Architecture

Date	04 JULY 2026
Team ID	SWTID-2026-2662
Project Name	AI-Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Solution Architecture Diagram:

Provide a high-level visual representation of your solution's architecture. Use the structural layout below as a template to map out your Presentation Layer, Application Layer, and Data Layer. Fill in the specific technologies and components for your project directly into the boxes to create your architectural diagram.



Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Manages the responsive, wide-layout multi-page interface (Dashboard, Financial Health, Predictor, Negotiation Email, Know Your Rights, History). It bypasses plain default widgets to render custom dark-themed HTML/CSS stat cards, badges, and progress bars.	Streamlit, Python 3.11+, HTML5, CSS3 (styles.css, components.py)
API Gateway	Exposes structured, asynchronous Restful API paths to receive front end network operations. It acts as an orchestrate that passes requests through validation filters, coordinates routers, and rejects unauthenticated client requests.	Fast API, Unicorn ASGI Server, Pedantic, Python requests library
Microservices	Houses all isolated back end logic engines. This includes token generation (120-min expiry), financial health computations (EM-to-income and stress scoring), settlement probability analysis, and the Google Gemini service paired with an offline, rule-based safe fallback generator.	Python Core Logic Engines (financial_engine.py, settlement_engine.py), Google Gemini API (gemini-1.5-flash), python-jose, Werkzeug
Database	Persists relational data models representing one-to-many user mappings (Profiles, Loans, Predictions, and Generation History records). It uses thread-concurrency properties to prevent transactional database locking during parallel client requests.	SQLite Database (finrelief.db), SQLAlchemy ORM mapping framework